TASK 7: From Procedural to Object-Oriented Programming

Name: Nagineni Sunil Kumar

Register Number: 192472330

1. Aim

To compare procedural and object-oriented implementations of the fare logic and demonstrate how encapsulation protects the smart-card balance.

2. Procedural Version

```
#include <iostream>

using namespace std;

const long long MIN_BALANCE = 5000;

bool canTravel(long long balance, long long fare) {

   return balance - fare >= MIN_BALANCE;}

bool deduct(long long& balance, long long fare) {

   if (fare < 0 || !canTravel(balance, fare))

      return false;

   balance -= fare;

   return true;}

int main() {

   long long balance = 10000;

   long long fare = 1000;

   if (deduct(balance, fare))

      cout << "Journey accepted\n";
```

```
    else
        cout << "Journey denied\n";
    cout << "Balance: " << balance << " paise\n";}
```

3. Object-Oriented Version

```
#include <iostream>
using namespace std;
class SmartCard {
private:
    long long balance;
    static constexpr long long MIN_BALANCE = 5000;
public:
    SmartCard(long long initialBalance) {
        balance = initialBalance >= 0 ? initialBalance : 0;   }
    long long getBalance() const {
        return balance;
    }
    bool canTravel(long long fare) const {
        return fare >= 0 && balance - fare >= MIN_BALANCE;  }
    bool deduct(long long fare) {
        if (!canTravel(fare))
            return false;
```

```
        balance -= fare;
        return true;    }};
int main() {
    SmartCard card(10000);
    if (card.deduct(1000))
        cout << "Journey accepted\n";
    else
        cout << "Journey denied\n";
    cout << "Balance: " << card.getBalance() << " paise\n";
}
```

4. Expected Output

Both programs produce:

Journey accepted

Balance: 9000 paise

Obtained output:-



5. Comparison

Feature	Procedural	Object-Oriented
Data storage	Separate variables	Stored inside objects
Data protection	Depends on function use	Private balance protects data
Code organisation	Functions	Classes and methods
Updating rules	May require changes across functions	Rules can be centralised in the class
Time per journey	O(1)	O(1)
Extra space per card	O(1)	O(1)

6. Why Make the Balance Private?

If the balance were public, another part of the program could assign an invalid value or deduct money without checking the minimum balance. Keeping the balance private forces updates through the class methods.

7. Alternative Concept and Complexity

Changing from a private to a public balance does not meaningfully change the time or space complexity. However, it removes the protection provided by encapsulation and makes it easier to violate the rule that at least ₹50 must remain after a journey.

8. Conclusion

Both styles can calculate fares correctly. The object-oriented design is preferable because it controls access to the balance and places the validation rules alongside the data they protect.